

Machine Learning II

Lecture 1: From the Perceptron to Backpropagation

Alexander Quispe
ENEI – INEI · PEU-CD 2026

October 2, 2026

Outline

Course Overview and Motivation

The Perceptron

From Perceptrons to Deep Networks

Introduction to PyTorch and Google Colab

Summary and Next Steps

Recap and Activation Functions Deep Dive

Loss Functions in Detail

The Chain Rule

Backpropagation Algorithm

Gradient Descent Variants

PyTorch Implementation

Summary and Next Steps

Course Overview & Motivation

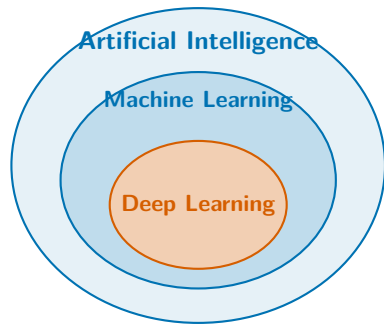
What is Deep Learning?

Machine Learning: algorithms that learn from data.

Deep Learning: a subset of ML that uses **neural networks with multiple layers** to learn hierarchical representations of data.

Key idea:

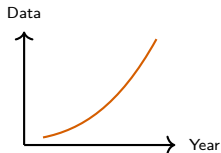
- Instead of hand-engineering features, the model *learns* the right representation.
- Each layer transforms the data into a more useful representation.
- **Depth** = number of stages of processing.



Why Deep Learning Now?

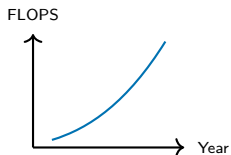
Three ingredients came together:

1. Data



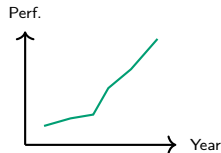
Internet, digitization, sensors,
satellites

2. Compute (GPUs)



NVIDIA GPUs, Google TPUs,
cloud computing

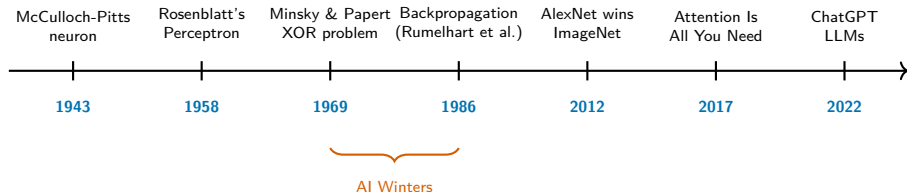
3. Algorithms



Backpropagation, ReLU,
dropout, Transformers

Neural networks existed since the 1950s — what changed is the scale.

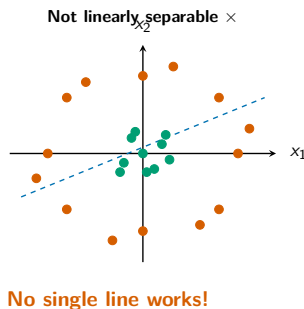
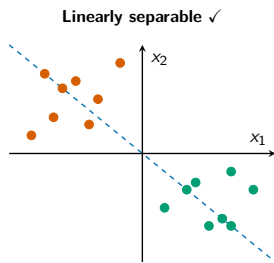
A Brief History of Neural Networks



- **1943–1969:** Early excitement → crushed by XOR limitation.
- **1986:** Backpropagation revives interest, but limited by compute.
- **2012:** Deep CNNs dominate computer vision (AlexNet).
- **2017–now:** Transformers → LLMs → foundation models revolution.

The Limitation of Linear Models

Linear models can only produce **linear decision boundaries**.



We need **non-linear** models. This is where neural networks come in.

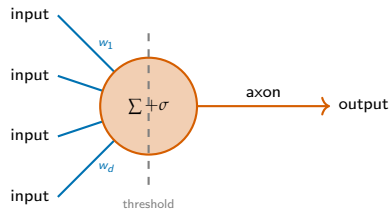
The Perceptron

Biological Inspiration

Biological neuron:

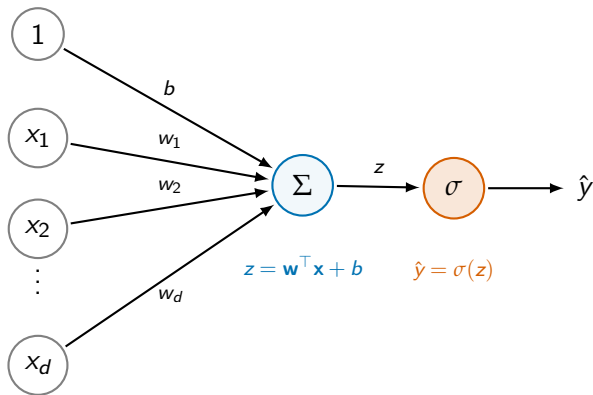
- Receives input signals from **dendrites**.
- Processes them in the **cell body**.
- If total signal exceeds a threshold, **fires** through the **axon**.
- Connections have different **strengths** (synaptic weights).

McCulloch & Pitts (1943) proposed the first mathematical model of a neuron.



The analogy is **loose**. Real neurons are far more complex. But the abstraction is powerful.

The Artificial Neuron (Perceptron)



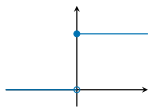
The Perceptron

$$\hat{y} = \sigma \left(\sum_{j=1}^d w_j x_j + b \right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

Activation Functions

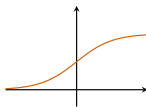
The activation function σ introduces **non-linearity** into the model.

Step (Heaviside)



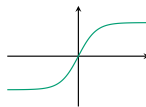
$$\sigma(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Sigmoid



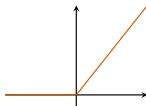
$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Tanh



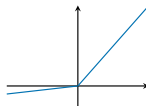
$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

ReLU (most popular)



$$\sigma(z) = \max(0, z)$$

Leaky ReLU



$$\sigma(z) = \max(0.01z, z)$$

Why Do We Need Non-linearity?

Critical Insight: Without activation functions, a multi-layer network collapses to a **single linear transformation**.

Proof: Consider two linear layers (no activation):

$$\begin{aligned} \mathbf{h}^{(1)} &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \\ \hat{\mathbf{y}} &= \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)} = \mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} \\ &= \underbrace{\mathbf{W}^{(2)}\mathbf{W}^{(1)}}_{\mathbf{W}'}\mathbf{x} + \underbrace{\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}}_{\mathbf{b}'} = \mathbf{W}'\mathbf{x} + \mathbf{b}' \end{aligned}$$

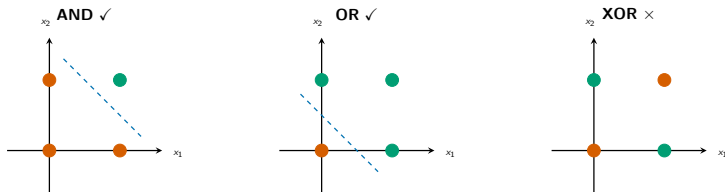
This is just **another linear model**! The depth is useless.

Non-linear activations are what make depth meaningful.

With activations: $\hat{\mathbf{y}} = \mathbf{W}^{(2)}\sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} \Rightarrow$ **genuinely non-linear**.

The XOR Problem: Why One Layer Is Not Enough

Boolean operations with a single perceptron:

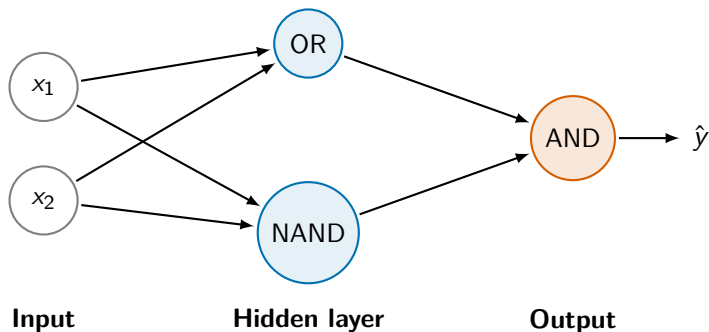


No single line!

- AND and OR are **linearly separable** — a single perceptron can learn them.
- XOR is **not linearly separable** — requires composing multiple linear boundaries.
- Shown by **Minsky & Papert (1969)** and caused the first “AI winter”.

Solving XOR with Two Layers

Key insight: $\text{XOR} = \text{AND}(\text{OR}(x_1, x_2), \text{NAND}(x_1, x_2))$



Adding one hidden layer solves the problem! This is the power of depth.

XOR: Truth Table Verification

The hidden layer computes OR and NAND; the output layer applies AND.

x_1	x_2	$\text{OR}(x_1, x_2)$	$\text{NAND}(x_1, x_2)$	$\text{AND}(\text{OR}, \text{NAND}) = \mathbf{XOR}$
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

- The hidden layer **transforms** the input into a new representation.
- In the new representation, the two classes **are** linearly separable.
- This is the essence of deep learning: **learn useful representations, then classify.**

XOR: The Explicit Computation

Using step activation $\sigma(z) = \mathbf{1}[z > 0]$:

Hidden layer:

$$\begin{aligned} h_1 &= \sigma(x_1 + x_2 - 0.5) && \text{(OR: fires if } x_1 + x_2 > 0.5) \\ h_2 &= \sigma(-x_1 - x_2 + 1.5) && \text{(NAND: fires if } x_1 + x_2 < 1.5) \end{aligned}$$

Output layer:

$$\hat{y} = \sigma(h_1 + h_2 - 1.5) \quad \text{(AND: fires if } h_1 + h_2 > 1.5)$$

In matrix form:

$$\mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} -0.5 \\ 1.5 \end{pmatrix}, \quad \mathbf{W}^{(2)} = (1 \quad 1), \quad b^{(2)} = -1.5$$

Verify for $(x_1, x_2) = (1, 0)$:

$$\begin{aligned} \mathbf{h} &= \sigma\left(\begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} -0.5 \\ 1.5 \end{pmatrix}\right) = \sigma\begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ \hat{y} &= \sigma(1 + 1 - 1.5) = \sigma(0.5) = 1 \quad \checkmark \end{aligned}$$

From Perceptrons to Deep Neural Networks

From One Neuron to a Layer

One neuron: computes a single output feature

$$h_j = \sigma \left(\sum_{k=1}^d w_{jk} x_k + b_j \right) = \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j)$$

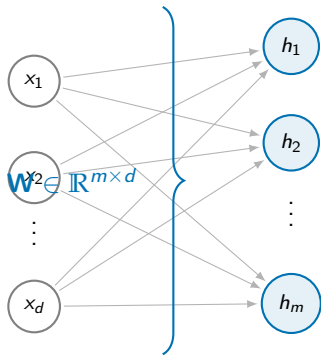
A layer of m neurons: computes m output features simultaneously

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) = \begin{pmatrix} \sigma(\mathbf{w}_1^\top \mathbf{x} + b_1) \\ \sigma(\mathbf{w}_2^\top \mathbf{x} + b_2) \\ \vdots \\ \sigma(\mathbf{w}_m^\top \mathbf{x} + b_m) \end{pmatrix} \in \mathbb{R}^m$$

where $\mathbf{W} \in \mathbb{R}^{m \times d}$ and $\mathbf{b} \in \mathbb{R}^m$.

Each neuron in the layer learns a **different linear boundary**; the activation function makes each one non-linear.

A Fully Connected Layer



Input (d features) Layer (m neurons)

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$$h_j = \sigma\left(\sum_{k=1}^d W_{jk} x_k + b_j\right)$$

Every input is connected to every neuron: this is called a **fully connected** (or **dense**) layer.
Parameters: $m \times d$ weights + m biases = $m(d + 1)$ total.

Stacking Layers: The Deep Neural Network

A **feedforward neural network** is a composition of layers:

Feedforward Neural Network with L layers

$$\mathbf{h}^{(0)} = \mathbf{x} \quad (\text{input})$$

$$\mathbf{h}^{(1)} = \sigma(\mathbf{W}^{(1)}\mathbf{h}^{(0)} + \mathbf{b}^{(1)}) \quad (\text{hidden layer 1})$$

$$\mathbf{h}^{(2)} = \sigma(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}) \quad (\text{hidden layer 2})$$

$$\vdots$$

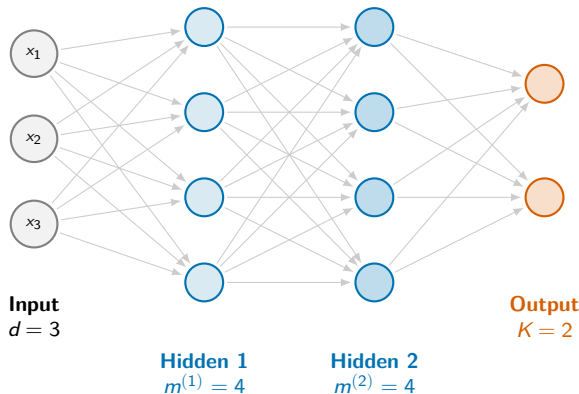
$$\mathbf{h}^{(L-1)} = \sigma(\mathbf{W}^{(L-1)}\mathbf{h}^{(L-2)} + \mathbf{b}^{(L-1)}) \quad (\text{hidden layer } L - 1)$$

$$\hat{\mathbf{y}} = \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)} \quad (\text{output layer})$$

Terminology:

- **Depth:** number of layers L (“deep” = $L \geq 2$ hidden layers).

Deep Neural Network: Diagram



Parameters:

$$\mathbf{W}^{(1)} : 4 \times 3 = 12$$

$$\mathbf{b}^{(1)} : 4$$

$$\mathbf{W}^{(2)} : 4 \times 4 = 16$$

$$\mathbf{b}^{(2)} : 4$$

$$\mathbf{W}^{(3)} : 2 \times 4 = 8$$

$$\mathbf{b}^{(3)} : 2$$

$$\text{Total: } 46$$

Modern networks: GPT-4 has ~ 1.8 **trillion** parameters. ResNet-50 has ~ 25 million.

Forward Pass: A Numerical Example

Network: 2 inputs $\rightarrow$ 2 hidden (ReLU) $\rightarrow$ 1 output (linear)

Given: $\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\mathbf{W}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{pmatrix}$, $\mathbf{b}^{(1)} = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$, $\mathbf{W}^{(2)} = (0.6 \quad -0.4)$,
 $b^{(2)} = 0.2$

Step 1: Compute hidden layer pre-activation

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} -0.1 \\ 1.7 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix} = \begin{pmatrix} 0.0 \\ 1.6 \end{pmatrix}$$

Step 2: Apply ReLU activation

$$\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = \begin{pmatrix} \max(0, 0.0) \\ \max(0, 1.6) \end{pmatrix} = \begin{pmatrix} 0.0 \\ 1.6 \end{pmatrix}$$

Step 3: Compute output

$$\hat{y} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + b^{(2)} = (0.6 \quad -0.4) \begin{pmatrix} 0.0 \\ 1.6 \end{pmatrix} + 0.2 = -0.64 + 0.2 = -0.44$$

Dimensions Cheat Sheet

For a network with layers of widths $d = m^{(0)}, m^{(1)}, m^{(2)}, \dots, m^{(L)}$:

Layer l	$\mathbf{W}^{(l)}$	$\mathbf{b}^{(l)}$	$\mathbf{h}^{(l)}$
Input ($l = 0$)	—	—	$\mathbb{R}^{m^{(0)}}$
Hidden l	$\mathbb{R}^{m^{(l)} \times m^{(l-1)}}$	$\mathbb{R}^{m^{(l)}}$	$\mathbb{R}^{m^{(l)}}$
Output ($l = L$)	$\mathbb{R}^{m^{(L)} \times m^{(L-1)}}$	$\mathbb{R}^{m^{(L)}}$	$\mathbb{R}^{m^{(L)}}$

Total parameters:

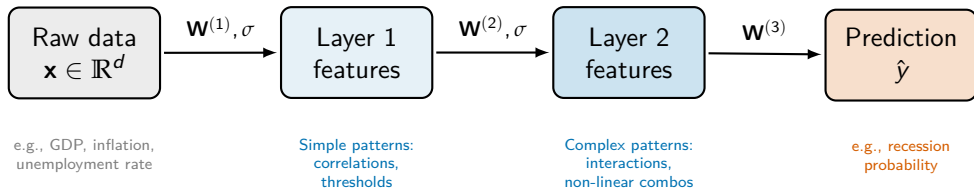
$$\sum_{l=1}^L \left(m^{(l)} \times m^{(l-1)} + m^{(l)} \right) = \sum_{l=1}^L m^{(l)} \left(m^{(l-1)} + 1 \right)$$

Example: Input 784 $\rightarrow$ Hidden 256 $\rightarrow$ Hidden 128 $\rightarrow$ Output 10

$$(256 \times 785) + (128 \times 257) + (10 \times 129) = 200,960 + 32,896 + 1,290 = 235,146$$

What Does a Neural Network Learn?

Key idea: each layer learns a **new representation** of the data.



For economists: think of it as **automatic feature engineering**:

- Instead of manually creating interaction terms ($x_1 \cdot x_2$, x_1^2 , etc.).
- The network **learns** which transformations of the raw features are useful.
- Particularly powerful when d is large (hundreds of variables).

Introduction to PyTorch & Google Colab

Why PyTorch?

PyTorch is the most widely used deep learning framework in research.

Key features:

- **Dynamic computation graphs** (define-by-run).
- **Automatic differentiation** (autograd).
- GPU acceleration (CUDA).
- Pythonic, intuitive API.
- Huge ecosystem (torchvision, torchtex, HuggingFace).

Alternatives:

- TensorFlow / Keras (Google).
- JAX (Google Research).
- Flux.jl (Julia).

We use PyTorch because:

- Standard in academia.
- Best for learning DL concepts.
- HuggingFace integration (for LLMs).

PyTorch Tensors

Tensors are the fundamental data structure — like NumPy arrays but with GPU support.

```
1 import torch
2
3 # Creating tensors
4 x = torch.tensor([1.0, 2.0, 3.0])           # 1D tensor (vector)
5 W = torch.randn(3, 2)                       # 2D tensor (matrix), random
6 b = torch.zeros(3)                          # 1D tensor of zeros
7
8 # Operations
9 z = W @ x + b                               # Matrix-vector multiply + bias (@ = matmul)
10 print(z.shape)                             # torch.Size([3])
11
12 # Move to GPU
13 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
14 x_gpu = x.to(device)
15 W_gpu = W.to(device)
16 z_gpu = W_gpu @ x_gpu                      # Computation happens on GPU!
```

Key difference from NumPy: tensors track computation for **automatic differentiation**.

Automatic Differentiation (Autograd)

PyTorch can automatically compute gradients — the foundation of training neural networks.

```
1 import torch
2
3 # Create tensors with gradient tracking
4 w = torch.tensor(2.0, requires_grad=True)
5 b = torch.tensor(1.0, requires_grad=True)
6 x = torch.tensor(3.0)
7 y_true = torch.tensor(7.0)      # target
8
9 # Forward pass: compute prediction and loss
10 y_pred = w * x + b              # y_pred = 2*3 + 1 = 7
11 loss = (y_true - y_pred) ** 2   # MSE for one sample = 0
12
13 # Backward pass: compute gradients
14 loss.backward()
15
16 # Gradients are now available!
17 print(w.grad)                   # d(loss)/dw = 2*(y_true - y_pred)*(-x) = 0
18 print(b.grad)                   # d(loss)/db = 2*(y_true - y_pred)*(-1) = 0
```

Key Insight: `loss.backward()` computes $\frac{\partial \mathcal{L}}{\partial w}$ and $\frac{\partial \mathcal{L}}{\partial b}$ **automatically** using the chain rule. We'll derive backpropagation in Lecture 2.

Implementing a Single Neuron in PyTorch

```
1 import torch
2 import torch.nn as nn
3
4 # A single neuron: 2 inputs -> 1 output with sigmoid activation
5 class SingleNeuron(nn.Module):
6     def __init__(self):
7         super().__init__()
8         self.linear = nn.Linear(2, 1)      # 2 inputs, 1 output
9         self.sigmoid = nn.Sigmoid()
10
11     def forward(self, x):
12         z = self.linear(x)                 #  $z = w^T x + b$ 
13         return self.sigmoid(z)             #  $\sigma(z)$ 
14
15 # Create the neuron and a sample input
16 neuron = SingleNeuron()
17 x = torch.tensor([[1.0, 2.0]])           # batch of 1, 2 features
18 y_pred = neuron(x)
19 print(f"Prediction: {y_pred.item():.4f}")
20
21 # Check parameters
22 for name, param in neuron.named_parameters():
23     print(f"{name}: {param.data}")
```

The Training Loop (Preview)

```
1 # The basic training loop structure
2 model = SingleNeuron()
3 optimizer = torch.optim.SGD(model.parameters(), lr=0.01) # SGD
4 criterion = nn.BCELoss() # Binary CE
5
6 for epoch in range(100):
7     # Forward pass
8     y_pred = model(x_train)
9     loss = criterion(y_pred, y_train)
10
11     # Backward pass
12     optimizer.zero_grad() # Reset gradients to zero
13     loss.backward() # Compute gradients (backpropagation)
14
15     # Update parameters
16     optimizer.step() # w <- w - alpha * grad
17
18     if epoch % 10 == 0:
19         print(f"Epoch {epoch}, Loss: {loss.item():.4f}")
```

Every DL training follows this pattern: forward → loss → backward → update.
We'll understand *exactly why* in Lecture 2 (backpropagation).

Google Colab Setup

Google Colab provides free access to GPUs in the browser.

Getting started:

1. Go to `colab.research.google.com`.
2. Create a new notebook.
3. Runtime → Change runtime type → **GPU**.
4. PyTorch is pre-installed!

Recommended subscriptions:

- **Colab Pro** (\$10/mo): better GPUs, longer runtime.
- **Colab Pro+** (\$50/mo): even more resources.
- Free tier works for Lectures 1–5.

Verify GPU access:

```
1 import torch
2 print(torch.cuda.is_available())
3 # True
4 print(torch.cuda.get_device_name(0))
5 # Tesla T4
```

Also recommended:

- **Claude Code** (\$20/mo) for coding assistance.
- Use LLMs as a learning tool, not a replacement.

Summary & Next Steps

Summary

1. **Supervised learning:** given data $\{(\mathbf{x}_i, y_i)\}$, find f that generalizes.
2. **Linear models** (regression/logistic) are the starting point:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b \quad \text{or} \quad p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

3. **Gradient descent** iteratively minimizes the loss:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}$$

4. **The perceptron** = linear model + activation function. Learns AND, OR but **not XOR**.
5. **Deep neural networks** stack layers of perceptrons:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

6. **Non-linear activations** (ReLU, sigmoid) are essential — without them, depth is useless.
7. **PyTorch** provides tensors + autograd for implementation.

Recap & Activation Functions

Activation Functions: Derivatives Matter

To do backpropagation, we need the **derivative** of each activation function.

Name	$\sigma(z)$	$\sigma'(z)$	Range
Sigmoid	$\frac{1}{1+e^{-z}}$	$\sigma(z)(1 - \sigma(z))$	$(0, 1)$
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$	$(-1, 1)$
ReLU	$\max(0, z)$	1 if $z > 0$; 0 if $z < 0$	$[0, \infty)$
Leaky ReLU	$\max(\alpha z, z)$	1 if $z > 0$; α if $z < 0$	$(-\infty, \infty)$

Deriving the Sigmoid Derivative

Let $\sigma(z) = \frac{1}{1+e^{-z}} = (1 + e^{-z})^{-1}$. Apply the chain rule:

Step 1: $\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2}$

Step 2: Rewrite in terms of $\sigma(z)$:

$$\sigma'(z) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z) \cdot \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z))$$

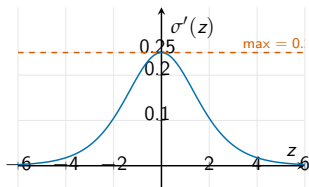
Result

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Note: Max value $\sigma'(0) = 0.25$. This causes the **vanishing gradient problem**.

The Vanishing Gradient Problem

Sigmoid derivative peaks at 0.25:



Problem for deep networks:

With L layers of sigmoid, the gradient at layer 1 involves:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} \propto \prod_{l=1}^L \sigma'(z^{(l)})$$

Since $\sigma'(z) \leq 0.25$:

$$\prod_{l=1}^L 0.25 = 0.25^L$$

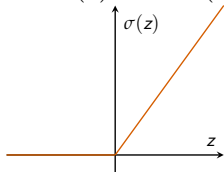
For $L = 10$: $0.25^{10} \approx 10^{-6}$

Gradients vanish exponentially with depth!

Solution: Use **ReLU**, where $\sigma'(z) \equiv 1$ for $z \geq 0$. No vanishing gradient!

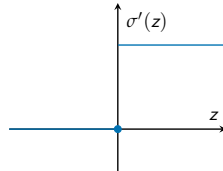
Why ReLU Dominates

ReLU: $\sigma(z) = \max(0, z)$



- Derivative = 1 for $z > 0$: no vanishing gradient.
- Computationally cheap.
- Sparse activation (many neurons output 0).

ReLU derivative:



Problem: “Dead ReLU” — if $z < 0$ always, the neuron never updates.

Fix: Leaky ReLU ($\alpha = 0.01$), ELU, GELU.

Softmax: From Binary to Multi-class Classification

For K classes, we need a probability distribution over classes.

Softmax Function

Given logits $z_1, z_2, \dots, z_K$ (raw outputs of the network), the **softmax** function computes:

$$p(y = k|\mathbf{x}) = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad \text{for } k = 1, \dots, K$$

Properties:

- $\sum_{k=1}^K p(y = k|\mathbf{x}) = 1$ (valid probability distribution).
- $p(y = k|\mathbf{x}) > 0$ for all k (always positive).
- Larger $z_k \Rightarrow$ higher probability for class k .
- When $K = 2$: softmax reduces to the sigmoid function.

Softmax: Numerical Example

Network output (logits) for $K = 3$ classes:

$$\mathbf{z} = \begin{pmatrix} z_1 \\ z_2 \\ z_3 \end{pmatrix} = \begin{pmatrix} 2.0 \\ 1.0 \\ 0.1 \end{pmatrix}$$

Step 1: Compute exponentials:

$$e^{z_1} = e^{2.0} = 7.389, \quad e^{z_2} = e^{1.0} = 2.718, \quad e^{z_3} = e^{0.1} = 1.105$$

Step 2: Sum of exponentials:

$$\sum_{j=1}^3 e^{z_j} = 7.389 + 2.718 + 1.105 = 11.212$$

Step 3: Normalize:

$$p_1 = \frac{7.389}{11.212} = 0.659, \quad p_2 = \frac{2.718}{11.212} = 0.242, \quad p_3 = \frac{1.105}{11.212} = 0.099$$

Prediction: class 1 with probability 65.9%. Check: $0.659 + 0.242 + 0.099 = 1.0$ ✓

Loss Functions in Detail

Loss Functions: Overview

The loss function $\mathcal{L}$ measures how far our prediction $\hat{y}$ is from the truth y .

Task	Loss	Formula	Output activation
Regression	MSE	$\frac{1}{N} \sum (y_i - \hat{y}_i)^2$	None (linear)
Binary classif.	BCE	$-\frac{1}{N} \sum [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$	Sigmoid
Multi-class	CE	$-\frac{1}{N} \sum \sum y_{ik} \log \hat{y}_{ik}$	Softmax

Key principle: the choice of loss function depends on the task. The loss determines what the network optimizes for.

MSE Loss: Gradient Derivation

Loss for one sample: $\ell = (y - \hat{y})^2$ where $\hat{y} = f(\mathbf{x}; \theta)$.

Gradient with respect to the prediction:

$$\frac{\partial \ell}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} (y - \hat{y})^2 = 2(y - \hat{y}) \cdot (-1) = -2(y - \hat{y})$$

Average over N samples:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -\frac{2}{N}(y_i - \hat{y}_i)$$

Interpretation:

- The gradient is proportional to the **error** $(y_i - \hat{y}_i)$.
- Large errors produce large gradients $\Rightarrow$ faster learning.
- This is the **starting point** of backpropagation: we first compute $\frac{\partial \mathcal{L}}{\partial \hat{y}}$, then propagate backward.

Binary Cross-Entropy: Derivation from MLE

Assume $y \in \{0, 1\}$ and model predicts $\hat{y} = p(y = 1|\mathbf{x})$. Likelihood of one sample:
 $p(y|\mathbf{x}) = \hat{y}^y(1 - \hat{y})^{1-y}$.

Log-likelihood $\rightarrow$ **Negative log-likelihood** (= BCE):

$$\ell = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

Gradient w.r.t. $\hat{y}$:

$$\frac{\partial \ell}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

Gradient w.r.t. logit z (where $\hat{y} = \sigma(z)$, $\sigma' = \sigma(1 - \sigma)$):

$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial \hat{y}} \cdot \sigma'(z) = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y}) = \hat{y} - y$$

The Beautiful Cancellation: BCE + Sigmoid

Key Result

$$\frac{\partial \ell}{\partial z} = \sigma(z) - y = \hat{y} - y$$

The gradient of BCE w.r.t. the logit z is simply $\hat{y} - y$.

Why is this beautiful?

- No $\sigma'(z)$ term in the gradient!
- Even when sigmoid saturates, gradient is large if prediction is wrong.
- **Not a coincidence:** BCE was designed to pair with sigmoid.
- Compare: MSE + sigmoid $\Rightarrow \frac{\partial \ell}{\partial z} = -2(y - \hat{y})\sigma'(z)$ – vanishes!

Lesson: Always pair the right loss with the right activation.

Categorical Cross-Entropy Loss

For K -class classification with one-hot encoding $\mathbf{y} = (y_1, \dots, y_K)$:

Categorical Cross-Entropy

$$\mathcal{L} = - \sum_{k=1}^K y_k \log \hat{y}_k = - \log \hat{y}_c$$

where c is the true class (since only $y_c = 1$, all other $y_k = 0$).

Example: True class = 2 (i.e., $\mathbf{y} = (0, 1, 0)$), predictions $\hat{\mathbf{y}} = (0.1, 0.7, 0.2)$:

$$\mathcal{L} = -(0 \cdot \log 0.1 + 1 \cdot \log 0.7 + 0 \cdot \log 0.2) = -\log(0.7) = 0.357$$

If predictions were $\hat{\mathbf{y}} = (0.1, 0.95, 0.05)$:

$$\mathcal{L} = -\log(0.95) = 0.051 \quad (\text{much lower — better prediction})$$

Interpretation: The loss only depends on the **predicted probability for the correct class**.

Softmax + Cross-Entropy: Gradient (Step 1)

Let $\hat{y}_k = \text{softmax}(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ and $\mathcal{L} = -\sum_k y_k \log \hat{y}_k$.

We want: $\frac{\partial \mathcal{L}}{\partial z_i}$ for each logit z_i . First, compute the **Jacobian of softmax**:

Case $i = k$:

$$\frac{\partial \hat{y}_k}{\partial z_k} = \frac{e^{z_k} \sum_j e^{z_j} - e^{z_k} \cdot e^{z_k}}{(\sum_j e^{z_j})^2} = \hat{y}_k - \hat{y}_k^2 = \hat{y}_k(1 - \hat{y}_k)$$

Case $i \neq k$:

$$\frac{\partial \hat{y}_k}{\partial z_i} = \frac{0 - e^{z_k} \cdot e^{z_i}}{(\sum_j e^{z_j})^2} = -\hat{y}_k \hat{y}_i$$

Compact form: $\frac{\partial \hat{y}_k}{\partial z_i} = \hat{y}_k(\delta_{ik} - \hat{y}_i)$ where δ_{ik} is the Kronecker delta.

Softmax + Cross-Entropy: Gradient (Step 2)

Step 2: Apply chain rule with $\mathcal{L} = -\sum_k y_k \log \hat{y}_k$:

$$\frac{\partial \mathcal{L}}{\partial z_i} = \sum_{k=1}^K \frac{\partial \mathcal{L}}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_i} = - \sum_{k=1}^K \frac{y_k}{\hat{y}_k} \cdot \hat{y}_k (\delta_{ik} - \hat{y}_i)$$

Simplify:

$$= - \sum_k y_k (\delta_{ik} - \hat{y}_i) = -y_i + \hat{y}_i \underbrace{\sum_k y_k}_{=1} = \hat{y}_i - y_i$$

Key Result

$$\frac{\partial \mathcal{L}}{\partial z_i} = \hat{y}_i - y_i \quad (\text{prediction minus truth})$$

Same result as BCE + sigmoid! This elegant cancellation is by design.

The Chain Rule

The Chain Rule: Single Variable

If $y = f(u)$ and $u = g(x)$, then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Example: Let $y = (3x + 2)^2$.

- Let $u = 3x + 2$, so $y = u^2$.
- $\frac{dy}{du} = 2u = 2(3x + 2)$.
- $\frac{du}{dx} = 3$.
- $\frac{dy}{dx} = 2(3x + 2) \cdot 3 = 6(3x + 2)$.

Longer chains: If $y = f(g(h(x)))$:

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$$

This is exactly what happens in a neural network: each layer is a function composition.

The Chain Rule: Multiple Variables

If $\mathcal{L} = f(y_1, y_2)$ where $y_1 = g_1(x)$ and $y_2 = g_2(x)$:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y_1} \frac{\partial y_1}{\partial x} + \frac{\partial \mathcal{L}}{\partial y_2} \frac{\partial y_2}{\partial x}$$

In general, if $\mathcal{L}$ depends on x through $y_1, y_2, \dots, y_m$:

$$\frac{\partial \mathcal{L}}{\partial x} = \sum_{j=1}^m \frac{\partial \mathcal{L}}{\partial y_j} \frac{\partial y_j}{\partial x}$$

Example (relevant to neural networks):

A weight $w_{13}^{(1)}$ in layer 1 affects multiple hidden neurons, which all affect the loss:

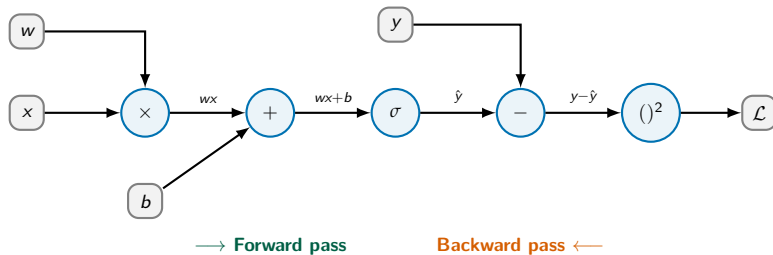
$$\frac{\partial \mathcal{L}}{\partial w_{13}^{(1)}} = \sum_j \frac{\partial \mathcal{L}}{\partial h_j^{(2)}} \cdot \frac{\partial h_j^{(2)}}{\partial w_{13}^{(1)}}$$

We need to sum contributions from all paths through the network.

Computational Graphs

A **computational graph** represents a function as a sequence of operations.

Example: $\mathcal{L} = (y - \sigma(wx + b))^2$



- **Forward pass:** compute output left to right.
- **Backward pass:** compute gradients right to left using chain rule.

Backward Pass Through the Graph

Starting from $\mathcal{L} = (y - \hat{y})^2$, let $e = y - \hat{y}$. Work right-to-left:

$$\frac{\partial \mathcal{L}}{\partial e} = 2e = 2(y - \hat{y}) \quad \text{(derivative of square)}$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial \mathcal{L}}{\partial e} \cdot \frac{\partial e}{\partial \hat{y}} = 2(y - \hat{y}) \cdot (-1) = -2(y - \hat{y}) \quad \text{(chain rule)}$$

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \sigma'(z) = -2(y - \hat{y}) \cdot \sigma'(z) \quad \text{(through activation)}$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot x = -2(y - \hat{y}) \sigma'(z) x \quad \text{(through multiplication)}$$

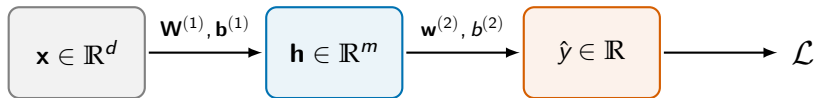
Key pattern: each node passes its gradient backward, multiplied by the local derivative.

Backpropagation

The engine of deep learning

Backpropagation: Setup

Consider a **2-layer network** (1 hidden layer):



Forward pass equations:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{h} = \sigma(\mathbf{z}^{(1)})$$

$$z^{(2)} = (\mathbf{w}^{(2)})^\top \mathbf{h} + b^{(2)}$$

$$\hat{y} = z^{(2)}$$

$$\mathcal{L} = (y - \hat{y})^2$$

$$\mathbf{z}^{(1)} \in \mathbb{R}^m \quad (\text{pre-activation})$$

$$\mathbf{h} \in \mathbb{R}^m \quad (\text{activation})$$

$$z^{(2)} \in \mathbb{R} \quad (\text{output logit})$$

(for regression, no final activation)

(MSE loss)

Goal: Compute $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}}$, $\frac{\partial \mathcal{L}}{\partial b^{(2)}}$.

Backpropagation Step 1: Output Layer Gradients

Start from the loss and work backward.

Step 1a: Gradient of loss w.r.t. output:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} (y - \hat{y})^2 = -2(y - \hat{y})$$

Since $\hat{y} = z^{(2)}$: $\frac{\partial \mathcal{L}}{\partial z^{(2)}} = -2(y - \hat{y})$. Define $\delta^{(2)} \equiv \frac{\partial \mathcal{L}}{\partial z^{(2)}} = -2(y - \hat{y})$.

Step 1b: Gradient w.r.t. output layer parameters:

$$z^{(2)} = \sum_{j=1}^m w_j^{(2)} h_j + b^{(2)}$$

$$\frac{\partial \mathcal{L}}{\partial w_j^{(2)}} = \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial w_j^{(2)}} = \delta^{(2)} \cdot h_j \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}} = \delta^{(2)} \cdot \mathbf{h}$$

$$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)}$$

Backpropagation Step 2: Hidden Layer Gradients

Step 2a: Propagate error back to hidden layer. Using chain rule:

$$\frac{\partial \mathcal{L}}{\partial h_j} = \delta^{(2)} \cdot \frac{\partial z^{(2)}}{\partial h_j} = \delta^{(2)} \cdot w_j^{(2)}$$

Step 2b: Pass through the activation function:

$$\frac{\partial \mathcal{L}}{\partial z_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial h_j} \cdot \sigma'(z_j^{(1)}) = \delta^{(2)} \cdot w_j^{(2)} \cdot \sigma'(z_j^{(1)}) \equiv \delta_j^{(1)}$$

Step 2c: Gradients w.r.t. hidden layer parameters (since $z_j^{(1)} = \sum_k W_{jk}^{(1)} x_k + b_j^{(1)}$):

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(1)}} = \delta_j^{(1)} \cdot x_k \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \delta^{(1)} \mathbf{x}^\top \quad \frac{\partial \mathcal{L}}{\partial b_j^{(1)}} = \delta_j^{(1)}$$

The General Backpropagation Formula

Define the **error signal** at layer l : $\delta^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$

Backpropagation Equations

Output layer ($l = L$): $\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} \odot \sigma'(\mathbf{z}^{(L)})$

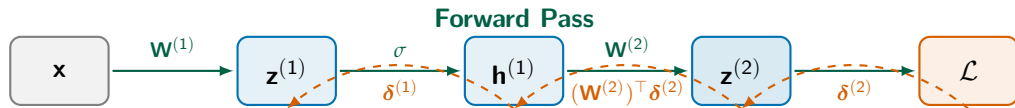
Hidden layers ($l = L-1, \dots, 1$): $\delta^{(l)} = \left[(\mathbf{W}^{(l+1)})^\top \delta^{(l+1)} \right] \odot \sigma'(\mathbf{z}^{(l)})$

Parameter gradients: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{h}^{(l-1)})^\top$ $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$

Here $\odot$ denotes element-wise (Hadamard) product and $\mathbf{h}^{(0)} = \mathbf{x}$.

Intuition: The error at layer l is the error from layer $l+1$ projected back through $\mathbf{W}^{(l+1)}$, scaled by how sensitive the activation was (σ').

Backpropagation: Visual Summary



Backward Pass (Backpropagation)

Algorithm:

1. **Forward:** Compute all $z^{(l)}$, $h^{(l)}$, and $\mathcal{L}$ (save intermediate values).
2. **Backward:** Compute $\delta^{(L)}, \delta^{(L-1)}, \dots, \delta^{(1)}$ using the recursion.
3. **Update:** $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha \delta^{(l)} (\mathbf{h}^{(l-1)})^\top$ for all l .

Backpropagation: Numerical Example (Part 1)

Network: 1 input $\rightarrow$ 1 hidden (sigmoid) $\rightarrow$ 1 output (linear). MSE loss.

Parameters: $w^{(1)} = 0.5$, $b^{(1)} = 0.1$, $w^{(2)} = -0.3$, $b^{(2)} = 0.2$

Data: $x = 2.0$, $y = 1.0$. **Learning rate:** $\alpha = 0.1$.

Forward pass:

$$z^{(1)} = w^{(1)}x + b^{(1)} = 0.5 \cdot 2.0 + 0.1 = 1.1$$

$$h = \sigma(z^{(1)}) = \sigma(1.1) = \frac{1}{1 + e^{-1.1}} = 0.7503$$

$$z^{(2)} = w^{(2)}h + b^{(2)} = -0.3 \cdot 0.7503 + 0.2 = -0.0251$$

$$\hat{y} = z^{(2)} = -0.0251$$

$$\mathcal{L} = (y - \hat{y})^2 = (1.0 - (-0.0251))^2 = (1.0251)^2 = 1.0508$$

Backpropagation: Numerical Example (Part 2)

Backward pass (work right-to-left through the network):

Step 1 – Error at output:

$$\delta^{(2)} = -2(y - \hat{y}) = -2(1.0 - (-0.0251)) = -2.0502$$

Step 2 – Output layer gradients:

$$\frac{\partial \mathcal{L}}{\partial w^{(2)}} = \delta^{(2)} \cdot h = (-2.0502)(0.7503) = -1.5383 \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)} = -2.0502$$

Step 3 – Error at hidden layer:

$$\delta^{(1)} = \delta^{(2)} \cdot w^{(2)} \cdot \sigma'(z^{(1)}) = (-2.0502)(-0.3)(0.7503)(1 - 0.7503) = 0.1154$$

Step 4 – Hidden layer gradients:

$$\frac{\partial \mathcal{L}}{\partial w^{(1)}} = \delta^{(1)} \cdot x = (0.1154)(2.0) = 0.2307 \quad \frac{\partial \mathcal{L}}{\partial b^{(1)}} = \delta^{(1)} = 0.1154$$

Backpropagation: Numerical Example (Part 3)

Parameter update with learning rate $\alpha = 0.1$: $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \text{grad}$

Parameter	Old	Gradient	New
$w^{(2)}$	-0.3000	-1.5383	-0.1462
$b^{(2)}$	0.2000	-2.0502	0.4050
$w^{(1)}$	0.5000	0.2307	0.4769
$b^{(1)}$	0.1000	0.1154	0.0885

Verify: Forward pass with new params $\rightarrow \hat{y}_{\text{new}} = 0.2969$

$$\mathcal{L}_{\text{new}} = (1.0 - 0.2969)^2 = 0.4946 \quad (\text{down from } 1.0508! \checkmark)$$

One gradient step reduced the loss by 53%. Repeat this process thousands of times to train the network.

Vector and Matrix Calculus Notation

In practice, we work with **matrices**, not individual weights.

Key identities (for layer l , with $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{h}^{(l-1)})^\top \quad \in \mathbb{R}^{m^{(l)} \times m^{(l-1)}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \quad \in \mathbb{R}^{m^{(l)}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(l-1)}} = (\mathbf{W}^{(l)})^\top \boldsymbol{\delta}^{(l)} \quad \in \mathbb{R}^{m^{(l-1)}}$$

Why outer product? Each $\delta_j^{(l)}$ pairs with each $h_k^{(l-1)}$: $\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} = \delta_j^{(l)} \cdot h_k^{(l-1)}$.

The gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$ has the same shape as $\mathbf{W}^{(l)}$.

Computational Cost of Backpropagation

- **Forward pass cost:** $O\left(\sum_{l=1}^L m^{(l)} m^{(l-1)}\right)$ (matrix-vector products).
- **Backward pass cost:** same order as forward pass.
- **Memory cost:** must store all $\mathbf{z}^{(l)}, \mathbf{h}^{(l)}$ during forward pass.
- **Total:** backprop is roughly **2x** the cost of the forward pass.

Historical Significance

Before backprop (1986): computing gradients for a network with P parameters required $O(P)$ separate forward passes.

With backprop: all gradients in one backward pass.

Gradient Descent Variants

Batch Gradient Descent

Full batch gradient descent uses **all** N samples to compute the gradient:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w})$$

Pros:

- Stable convergence, low noise.
- Exact gradient direction.

Cons:

- Requires loading **entire dataset** into memory.
- Very slow for large N (one update per epoch).
- Can get stuck in local minima (smooth trajectory).

Not practical when N is large (e.g., ImageNet: $N = 1.2$ million).

Stochastic Gradient Descent (SGD)

SGD uses a **single random sample** per update:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w}) \quad \text{where } i \text{ is sampled randomly}$$

Pros:

- Very fast updates.
- Can escape shallow local minima (noise helps!).
- Low memory requirement.

Cons:

- Very noisy gradient estimate.
- Oscillates around minimum.
- May never converge exactly.

Key insight: SGD's gradient is an **unbiased estimate** of the true gradient:

$$\mathbb{E}_i [\nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w})] = \frac{1}{N} \sum_{i=1}^N \nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w}) = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

Mini-Batch SGD: The Best of Both Worlds

Mini-batch SGD uses a random subset $\mathcal{B} \subset \{1, \dots, N\}$ of size B :

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\mathbf{w}} \ell(\mathbf{x}_i, y_i; \mathbf{w})$$

Method	Batch size	Noise	Speed
Batch GD	$B = N$	None	Slow
SGD	$B = 1$	High	Fast
Mini-batch SGD	$B = 32, 64, 128, 256$	Moderate	Fast

Why mini-batches work so well:

- GPUs handle parallel matmul — 32 samples $\approx$ as fast as 1.
- Noise reduced by factor $\frac{1}{\sqrt{B}}$ vs. pure SGD.
- Typical: $B = 32, 64, 128, 256$ (powers of 2 for GPU efficiency).

Epochs, Iterations, and Batch Size

Terminology

- **Epoch:** one complete pass through the entire dataset.
- **Iteration:** one parameter update (= one mini-batch).
- **Iterations per epoch:** $\lceil N/B \rceil$.

Example: $N = 10,000$ training samples, batch size $B = 100$:

- Iterations per epoch: $10,000/100 = 100$.
- If we train for 50 epochs: $50 \times 100 = 5,000$ total parameter updates.

Training recipe:

1. Shuffle the dataset.
2. Split into mini-batches of size B .
3. For each mini-batch: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update.
4. Repeat for multiple epochs.

SGD with Momentum

Pure SGD oscillates due to noisy gradients. **Momentum** smooths the trajectory:

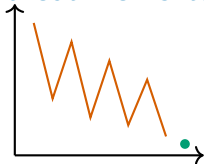
Momentum Update

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t) \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{v}_t$$

where $\beta \in [0, 1)$ is the momentum coefficient (typically $\beta = 0.9$).

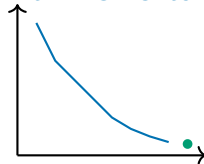
Intuition: a ball rolling down a hill. Momentum accumulates past gradients, so it “rolls through” small bumps.

Without momentum



Oscillates, slow convergence.

With momentum



Smoother, faster convergence.

PyTorch Implementation

Backpropagation in PyTorch: Under the Hood

PyTorch does backpropagation automatically via autograd.

```
1 import torch
2 import torch.nn as nn
3
4 # 2-layer network: 3 inputs -> 4 hidden (ReLU) -> 1 output
5 model = nn.Sequential(
6     nn.Linear(3, 4),      # W1: 4x3, b1: 4 -> 16 params
7     nn.ReLU(),
8     nn.Linear(4, 1))      # W2: 1x4, b2: 1 -> 5 params (Total: 21)
9
10 # Forward pass + loss
11 x = torch.randn(1, 3); y_true = torch.tensor([[1.0]])
12 loss = nn.MSELoss()(model(x), y_true)
13
14 # Backward pass -- ALL gradients computed automatically!
15 loss.backward()
16
17 for name, param in model.named_parameters():
18     print(f"{name}: grad shape = {param.grad.shape}")
```


Full Training Example: Economic Data

```
1 import torch
2 import torch.nn as nn
3 from torch.utils.data import DataLoader, TensorDataset
4
5 # Simulated economic data: 3 features -> outcome
6 X = torch.randn(1000, 3)
7 y = (2*X[:,0] - X[:,1] + 0.5*X[:,2] + torch.randn(1000)*0.1).unsqueeze(1)
8
9 # Mini-batch DataLoader
10 loader = DataLoader(TensorDataset(X, y), batch_size=32, shuffle=True)
11
12 # Model, loss, optimizer
13 model = nn.Sequential(nn.Linear(3, 16), nn.ReLU(),
14                       nn.Linear(16, 8), nn.ReLU(), nn.Linear(8, 1))
15 criterion = nn.MSELoss()
16 optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
17
18 # Training loop
19 for epoch in range(50):
20     for X_batch, y_batch in loader:
21         loss = criterion(model(X_batch), y_batch)
22         optimizer.zero_grad(); loss.backward(); optimizer.step()
```

Visualizing the Training Loss

```
1 import matplotlib.pyplot as plt
2
3 losses = []
4 for epoch in range(100):
5     epoch_loss = 0
6     for X_batch, y_batch in loader:
7         loss = criterion(model(X_batch), y_batch)
8         optimizer.zero_grad(); loss.backward(); optimizer.step()
9         epoch_loss += loss.item()
10    losses.append(epoch_loss / len(loader))
11
12 plt.plot(losses); plt.xlabel('Epoch'); plt.ylabel('MSE Loss')
13 plt.title('Training Loss Curve'); plt.grid(True); plt.show()
```

What to look for: loss should decrease; oscillations $\Rightarrow$ LR too large; slow decrease $\Rightarrow$ LR too small.

Summary & Next Steps

Summary

1. **Activation function derivatives** are critical for backprop. Sigmoid: $\sigma' = \sigma(1 - \sigma)$.
ReLU: $\sigma' = \mathbf{1}[z > 0]$.
2. **Vanishing gradients**: sigmoid derivative ≤ 0.25 causes exponential decay in deep nets.
ReLU solves this.
3. **Softmax** extends binary classification to K classes: $\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$.
4. **Cross-entropy + softmax/sigmoid** gives gradient $\frac{\partial \mathcal{L}}{\partial z_i} = \hat{y}_i - y_i$.
5. **Backpropagation** = chain rule applied layer by layer:

$$\delta^{(l)} = [(\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}] \odot \sigma'(\mathbf{z}^{(l)}) \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{h}^{(l-1)})^\top$$

6. **Mini-batch SGD** with momentum is the standard optimizer. Typical $B = 32$ – 256 .

Next Lecture: SGD, Regularization & Generalization

Lecture 3 (Monday, April 20):

- Advanced optimizers: Adam, AdaGrad, RMSProp.
- Regularization: L1, L2 (weight decay), dropout.
- Batch normalization.
- Universal Approximation Theorem; why depth matters.
- Practical tips: LR schedules, early stopping.

Recommended reading: Goodfellow et al., *Deep Learning*, Ch. 6.5 + Ch. 8. Zhang et al., *D2L*, Ch. 5 (<https://d2l.ai>).

Homework 1 will be released after Lecture 3 (due Sunday, April 27).

Questions?

References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapters 6, 8.
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*.
- Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. *ICML*.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*.